Detailed Evidence Report: UNSW-NB15 Intrusion Detection

Lab 01 — AI for Cybersecurity
Prepared by: **Ahmed Mohamed Houssam**
Date: 12 October 2026

> **Evidence basis.** This report is based on the 17 Google Colab screenshots in `manual-screeshots/`. Values below are transcribed from those captures, especially screenshots 7, 10, 14–17.

Executive summary

The notebook implements a leakage-safe intrusion-detection workflow on the supplied UNSW-NB15 data. Random Forest is the strongest observed model for ten-class attack attribution (macro F1 **0.4660**) and Normal-versus-Attack detection (macro F1 **0.8649**). For rare Worms detection, balanced Random Forest is the strongest treatment (macro F1 **0.9285**).

1. Data audit and feature policy

The training/test sets contain 175,341/82,332 flows with 45 columns. The audit found no missing cells, numeric infinity, duplicate rows, or inconsistencies between `attack_cat` and `label`. The training distribution is imbalanced: 56,000 Normal flows but only 130 Worms flows. The model keeps 42 predictors (39 numeric, 3 categorical) and removes `id`, `attack_cat`, and `label`. The identifier is not behaviour, and both label columns must never appear in model inputs.

2. Method and anti-leakage design

Only the supplied training file is split, using a 90/10 stratified development/validation split. The supplied test file is untouched until final evaluation. Each scikit-learn pipeline fits median imputation and StandardScaler for numeric data, plus most-frequent imputation and one-hot encoding for categorical data, using training data only. Logistic Regression is the linear baseline; Random Forest learns non-linear decision rules. Macro F1 selects the small validation search because every class, including rare Worms, should count equally.

3. Validation and overfitting

The unlimited-depth multiclass forest has train macro F1 0.7979 and validation macro F1 0.5710: the 0.2270 gap warns of overfitting. It is still selected because it has the best multiclass validation macro F1 among the tested forests. In Normal-versus-Attack, the forest gap is smaller (0.0438), suggesting more stable generalisation. The held-out test results below are the final estimate of generalisation.

4. Final held-out test results

Experiment	Model / setting	Accuracy	Macro P	Macro R	Macro F1	ROC-AUC
Attack category	Logistic Regression, C=1.0	.6981	.3806	.3847	.3365	.9488*
Attack category	Random Forest, unlimited	.7543	.5372	.4846	**.4660**	.9420*
Normal vs Attack	Logistic Regression, C=.3	.8076	.8492	.7891	.7933	.9550
Normal vs Attack	Random Forest, unlimited	.8711	.8971	.8582	**.8649**	.9786
Normal vs Worms	Logistic Regression, unweighted	.9994	.8873	.8522	.8689	.9817
Normal vs Worms	Logistic Regression, balanced	.9683	.5156	.9160	.5223	.9794
Normal vs Worms	Random Forest, unweighted	.9996	.9486	.8977	.9216	.9993
Normal vs Worms	Random Forest, balanced	.9997	.9499	.9090	**.9285**	.9986

*One-versus-rest macro ROC-AUC for the ten-class task.

Experiment 1 — Attack-category attribution

Random Forest is better than Logistic Regression in macro F1 (0.4660 versus 0.3365). The confusion matrix shows that this remains a difficult task for rare classes; only 5 of 44 Worms flows are correctly classified as Worms. Therefore weighted averages and accuracy must not be used alone.

Experiment 2 — Normal versus Attack

Random Forest correctly detects 44,685 attacks and misses 647; it raises 9,968 alerts on normal flows. This gives attack recall 0.9857 and Normal recall 0.7306. False negatives are attacks that escape detection; false positives consume analyst time. The right operating point depends on security risk and analyst capacity.

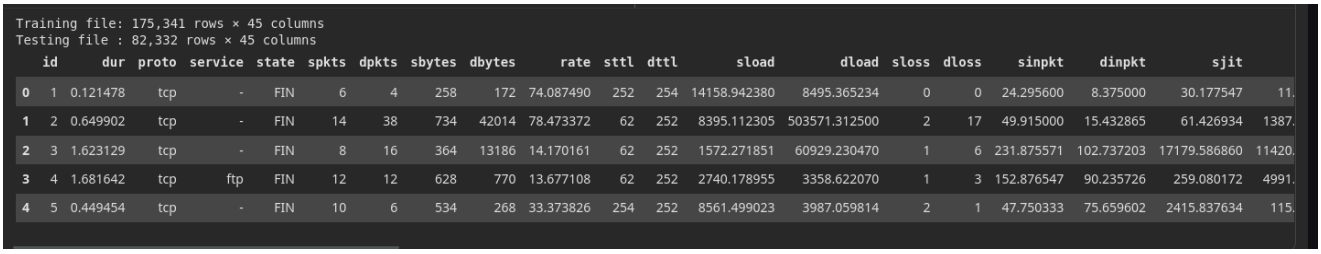
Experiment 3 — Normal versus Worms

The test subset contains 37,000 Normal but only 44 Worms flows. Balanced Random Forest finds 36 Worms and misses 8, compared with 35 found and 9 missed without weighting; both versions create 4 normal false positives. In contrast, balanced Logistic Regression finds 38 Worms but creates 1,170 normal false positives, which explains its much lower macro F1. Class weighting is a trade-off, not a magic fix.

5. Conclusion

Random Forest is recommended for both principal tasks, and its balanced version is preferred for the rare Worms stress test. The results are valid for this benchmark but should be rechecked on newer, local, temporally separate traffic before any real deployment.

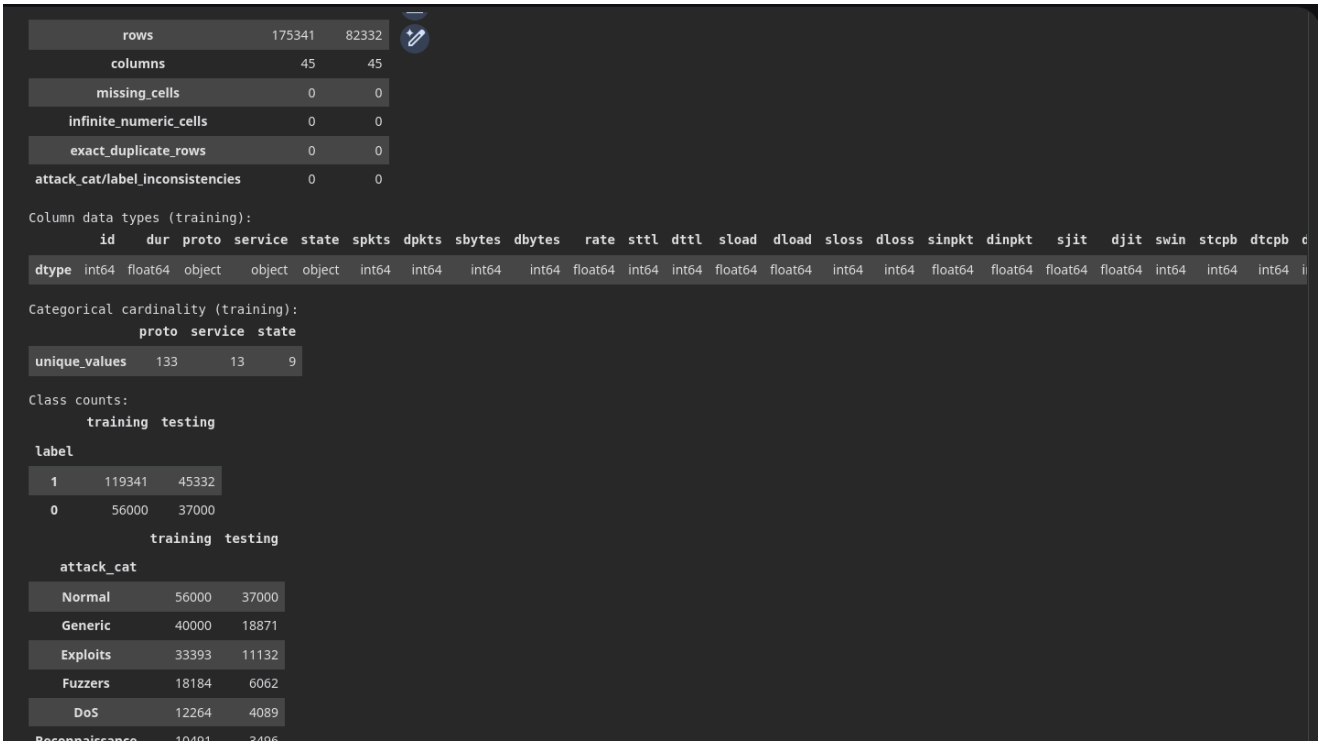
Appendix — complete Google Colab screenshot evidence



```
Training file: 175,341 rows × 45 columns
Testing file : 82,332 rows × 45 columns
```

	id	dur	proto	service	state	spkts	dpkts	sbytes	dbytes	rate	sttl	dttl	sload	dload	sloss	dloss	sinpkt	dinpkt	sjit	
0	1	0.121478	tcp	-	FIN	6	4	258	172	74.087490	252	254	14158.942380	8495.365234	0	0	24.295600	8.375000	30.177547	11.
1	2	0.649902	tcp	-	FIN	14	38	734	42014	78.473372	62	252	8395.112305	503571.312500	2	17	49.915000	15.432865	61.426934	1387.
2	3	1.623129	tcp	-	FIN	8	16	364	13186	14.170161	62	252	1572.271851	60929.230470	1	6	231.875571	102.737203	17179.586860	11420.
3	4	1.681642	tcp	ftp	FIN	12	12	628	770	13.677108	62	252	2740.178955	3358.622070	1	3	152.876547	90.235726	259.080172	4991.
4	5	0.449454	tcp	-	FIN	10	6	534	268	33.373826	254	252	8561.499023	3987.059814	2	1	47.750333	75.659602	2415.837634	115.

Screenshot 1 — Loading output and first five training records.



rows	175341 82332
columns	45 45
missing_cells	0 0
infinite_numeric_cells	0 0
exact_duplicate_rows	0 0
attack_cat/label_inconsistencies	0 0

Column data types (training):

	id	dur	proto	service	state	spkts	dpkts	sbytes	dbytes	rate	sttl	dttl	sload	dload	sloss	dloss	sinpkt	dinpkt	sjit	djit	swin	stcpb	dtcpb	d
dtype	int64	float64	object	object	object	int64	int64	int64	int64	float64	int64	int64	float64	float64	int64	int64	float64	float64	float64	float64	int64	int64	int64	i

Categorical cardinality (training):

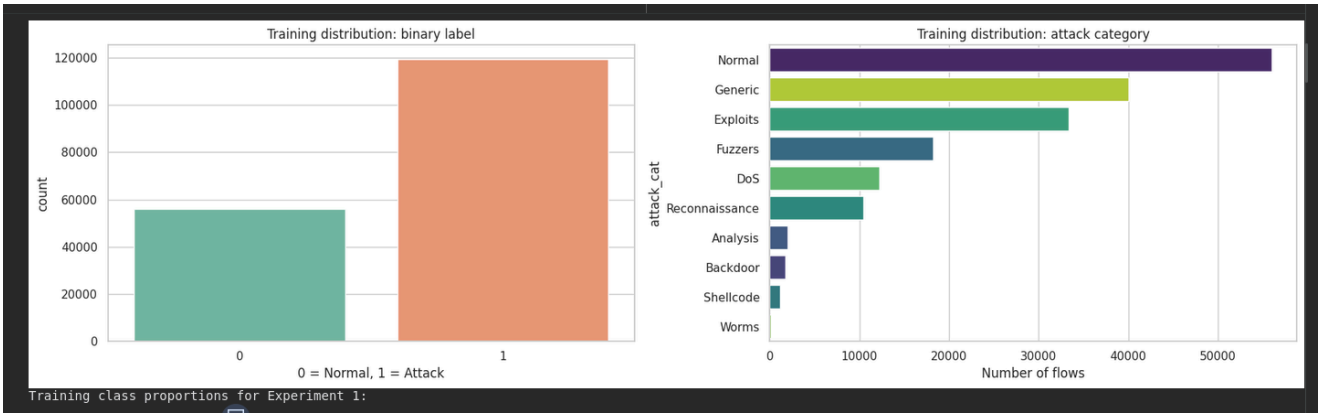
	proto	service	state
unique_values	133	13	9

Class counts:

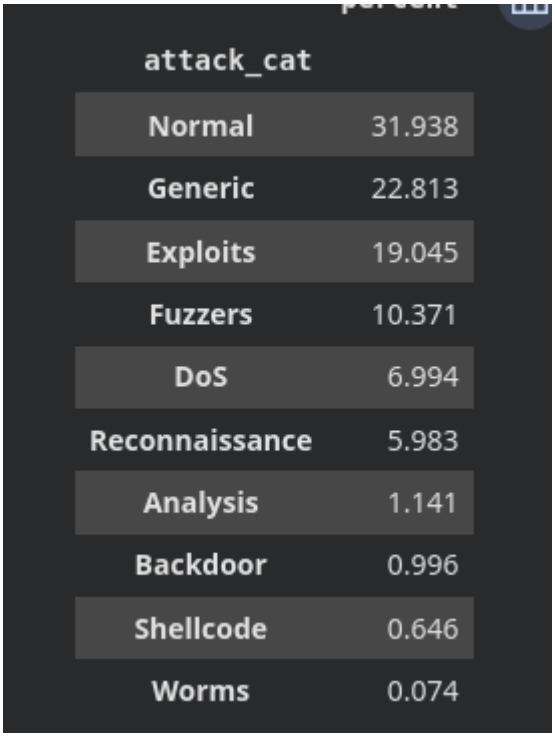
label	training	testing
1	119341	45332
0	56000	37000

attack_cat	training	testing
Normal	56000	37000
Generic	40000	18871
Exploits	33393	11132
Fuzzers	18184	6062
DoS	12264	4089

Screenshot 2 — Audit, data types, categorical cardinality, and class counts.



Screenshot 3 — Binary and ten-class training distributions.

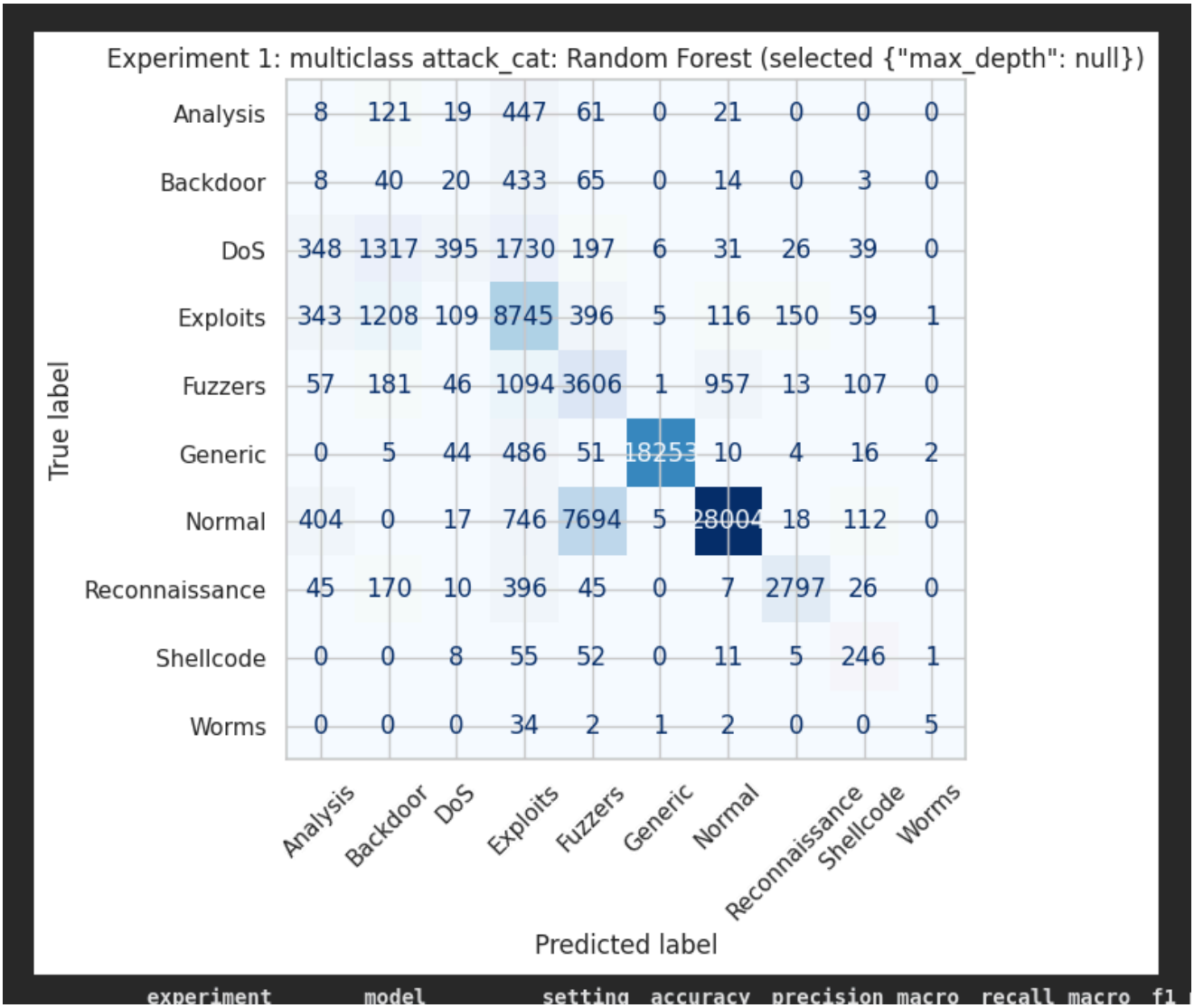


attack_cat	
Normal	31.938
Generic	22.813
Exploits	19.045
Fuzzers	10.371
DoS	6.994
Reconnaissance	5.983
Analysis	1.141
Backdoor	0.996
Shellcode	0.646
Worms	0.074

Screenshot 4 — Attack-category percentages, including rare Worms.

```
Predictors: 42 = 39 numeric + 3 categorical
Dropped: ['id', 'attack_cat', 'label']
```

Screenshot 5 — Final predictor policy: 42 predictors and dropped columns.



Screenshot 6 — Experiment 1 Random Forest confusion matrix.

	experiment	model	setting	accuracy	precision_macro	recall_macro	f1_macro	precision_weighted	recall_weighted	f1_weighted	roc_auc_ovr_macro
0	Experiment 1: multiclass attack_cat	Logistic Regression	selected {"C": 1.0}	0.6981	0.3806	0.3847	0.3365	0.7945	0.6981	0.7060	0.9488
1	Experiment 1: multiclass attack_cat	Random Forest	selected {"max_depth": null}	0.7543	0.5372	0.4846	0.4660	0.8367	0.7543	0.7756	0.9420

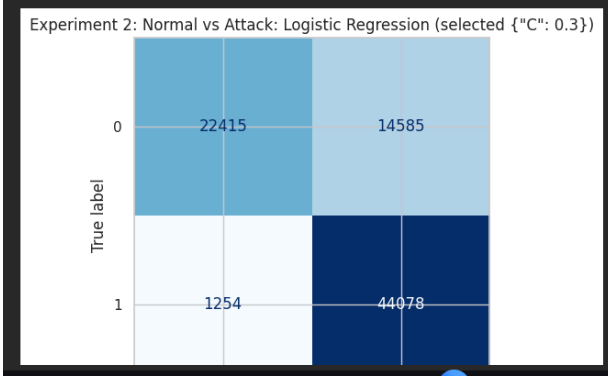
Screenshot 7 — Experiment 1 final metrics table.

	experiment	model	parameters	train_macro_f1	validation_macro_f1	validation_accuracy	validation_roc_auc	overfit_gap_tr	n
0	Experiment 2: Normal vs Attack	Logistic Regression	{"C": 0.3}	0.9211	0.9242	0.9369	0.9838	-0.0031	
1	Experiment 2: Normal vs Attack	Logistic Regression	{"C": 1.0}	0.9221	0.9241	0.9368	0.9840	-0.0021	
3	Experiment 2: Normal vs Attack	Random Forest	{"max_depth": null}	0.9980	0.9542	0.9604	0.9939	0.0438	
2	Experiment 2: Normal vs Attack	Random Forest	{"max_depth": 20}	0.9766	0.9505	0.9575	0.9938	0.0261	

```
Experiment 2: Normal vs Attack | Logistic Regression | selected {"C": 0.3}
              precision    recall  f1-score   support

           0     0.9470    0.6058    0.7389     37000
           1     0.7514    0.9723    0.8477     45332

    accuracy                         0.8076     82332
   macro avg     0.8492    0.7891    0.7933     82332
weighted avg     0.8393    0.8076    0.7988     82332
```

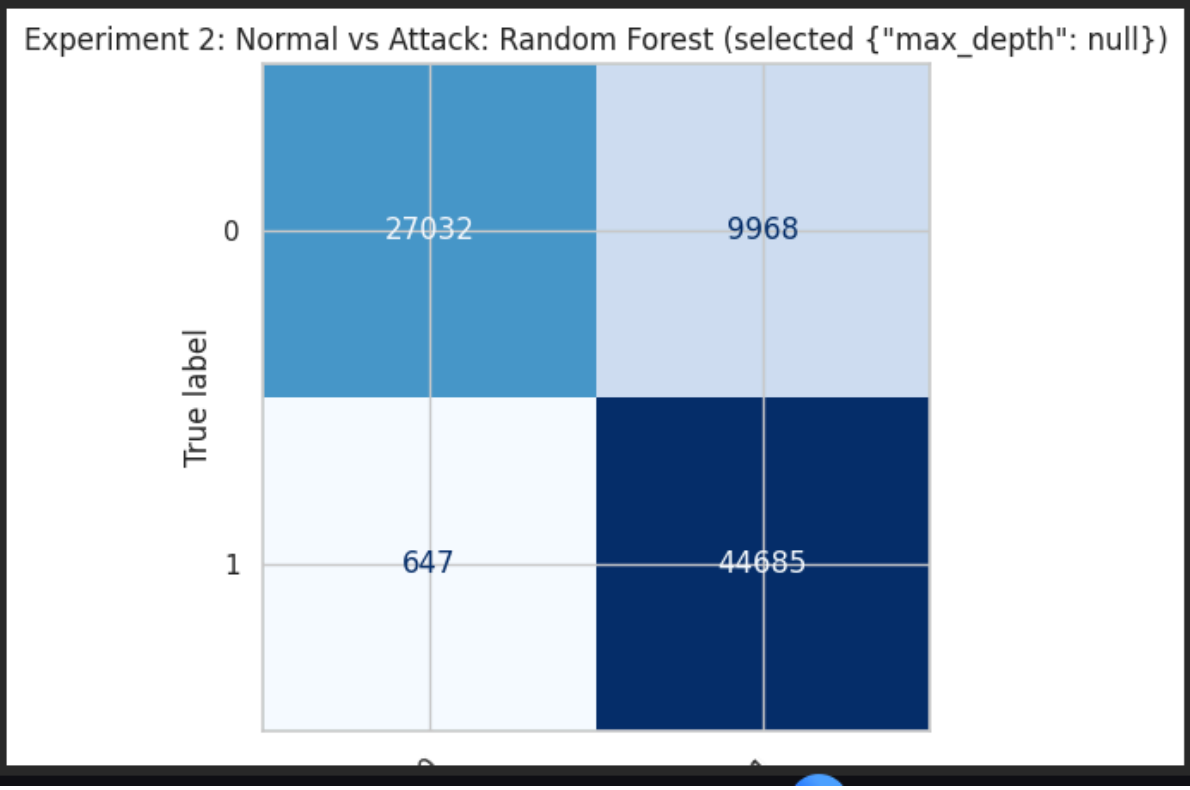


Screenshot 8 — Experiment 2 Logistic Regression report and confusion matrix.

```
Experiment 2: Normal vs Attack | Random Forest | selected {"max_depth": null}
              precision    recall  f1-score   support

           0     0.9766    0.7306    0.8359     37000
           1     0.8176    0.9857    0.8938     45332

    accuracy                         0.8711     82332
   macro avg     0.8971    0.8582    0.8649     82332
weighted avg     0.8891    0.8711    0.8678     82332
```



Screenshot 9 — Experiment 2 Random Forest report and confusion matrix.

	experiment	model	setting	accuracy	precision_macro	recall_macro	f1_macro	precision_weighted	recall_weighted	f1_weighted	roc_auc
0	Experiment 2: Normal vs Attack	Logistic Regression	selected {"C": 0.3}	0.8076	0.8492	0.7891	0.7933	0.8393	0.8076	0.7988	0.9550
1	Experiment 2: Normal vs Attack	Random Forest	selected {"max_depth": null}	0.8711	0.8971	0.8582	0.8649	0.8891	0.8711	0.8678	0.9786

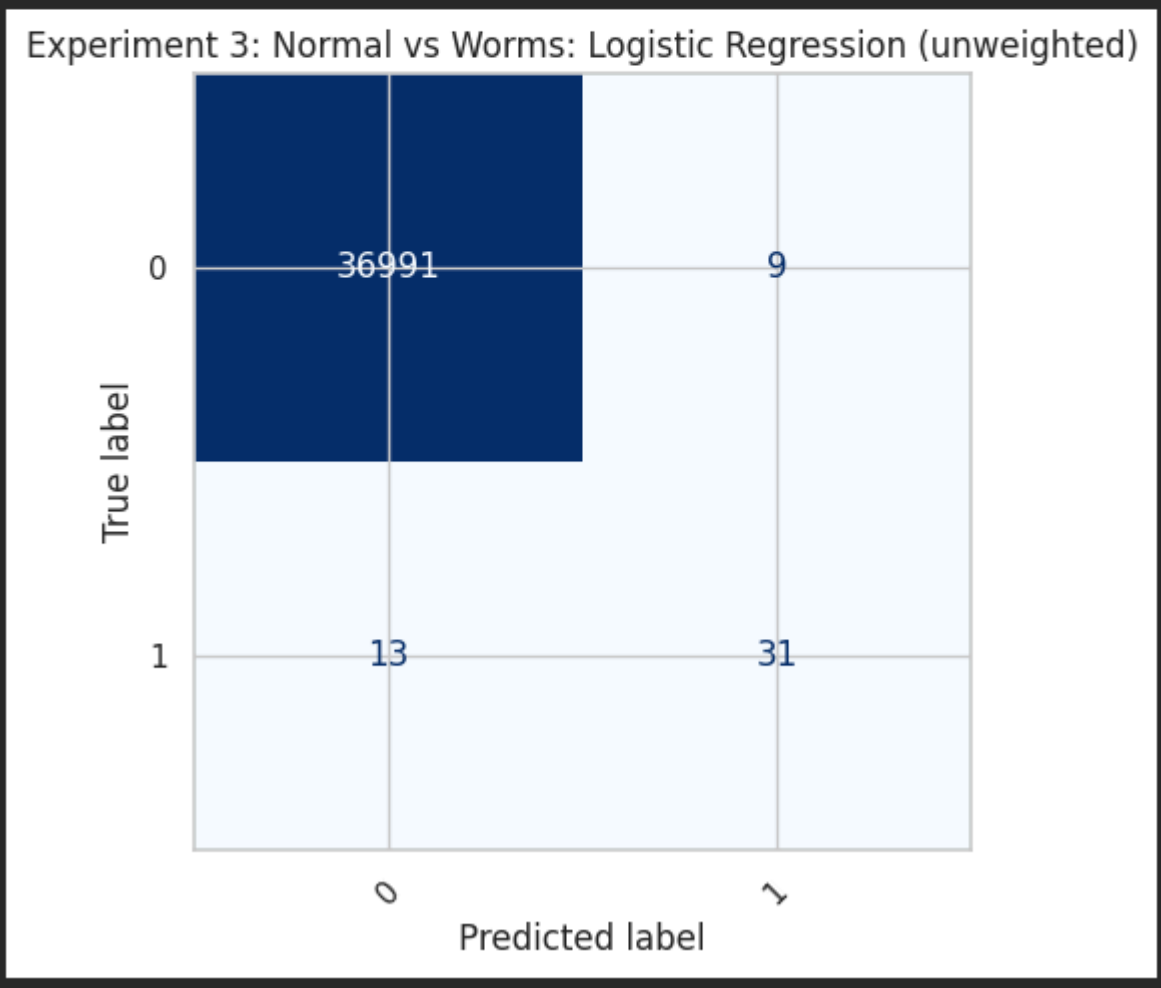
Screenshot 10 — Experiment 2 final comparison table.

```
Normal/Worms training counts: {0: 56000, 1: 130}
Normal/Worms testing counts : {0: 37000, 1: 44}

Experiment 3: Normal vs Worms | Logistic Regression | unweighted
              precision    recall  f1-score   support

           0     0.9996    0.9998    0.9997     37000
           1     0.7750    0.7045    0.7381        44

    accuracy                         0.9994     37044
   macro avg     0.8873    0.8522    0.8689     37044
weighted avg     0.9994    0.9994    0.9994     37044
```

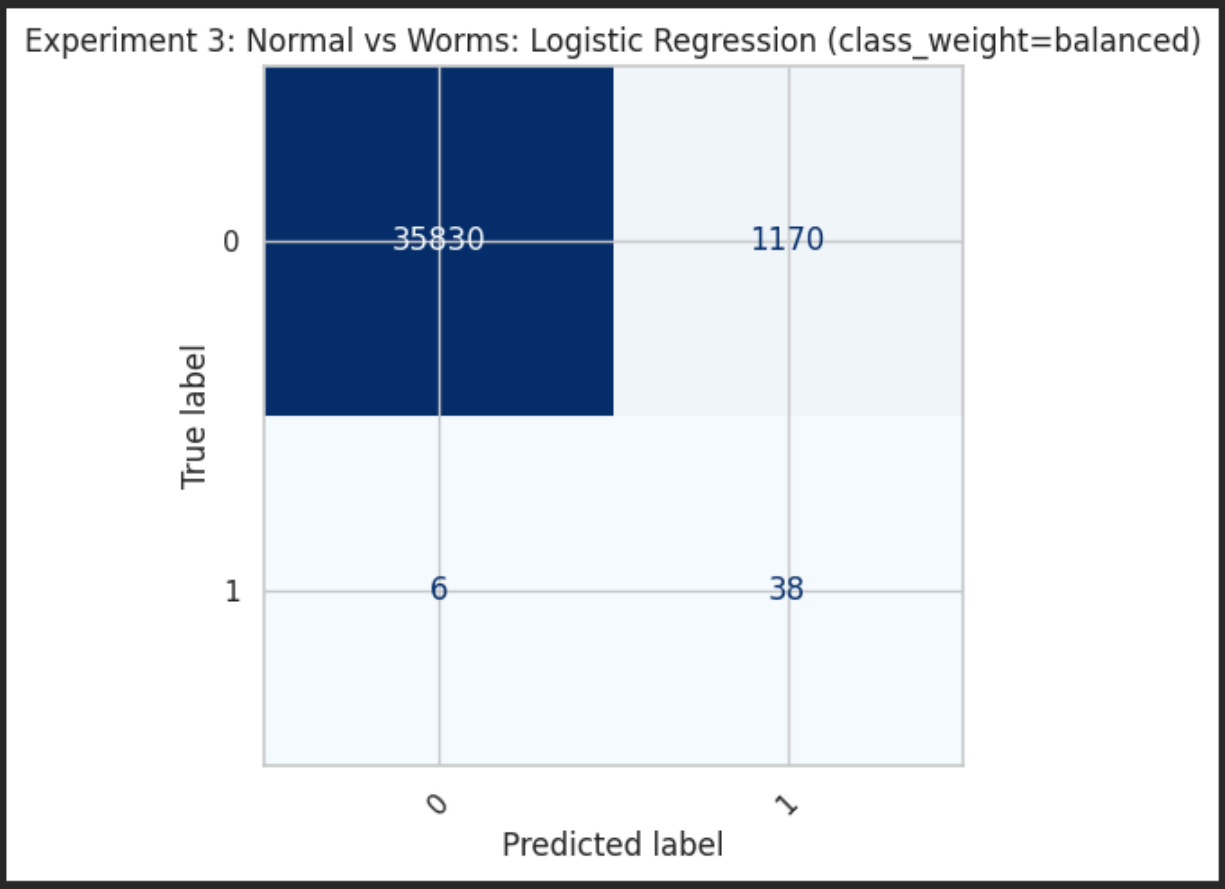


Screenshot 11 — Experiment 3 unweighted Logistic Regression.

```
Experiment 3: Normal vs Worms | Logistic Regression | class_weight=balanced
              precision    recall  f1-score   support

           0     0.9998    0.9684    0.9839     37000
           1     0.0315    0.8636    0.0607        44

    accuracy                         0.9683     37044
   macro avg     0.5156    0.9160    0.5223     37044
weighted avg     0.9987    0.9683    0.9828     37044
```

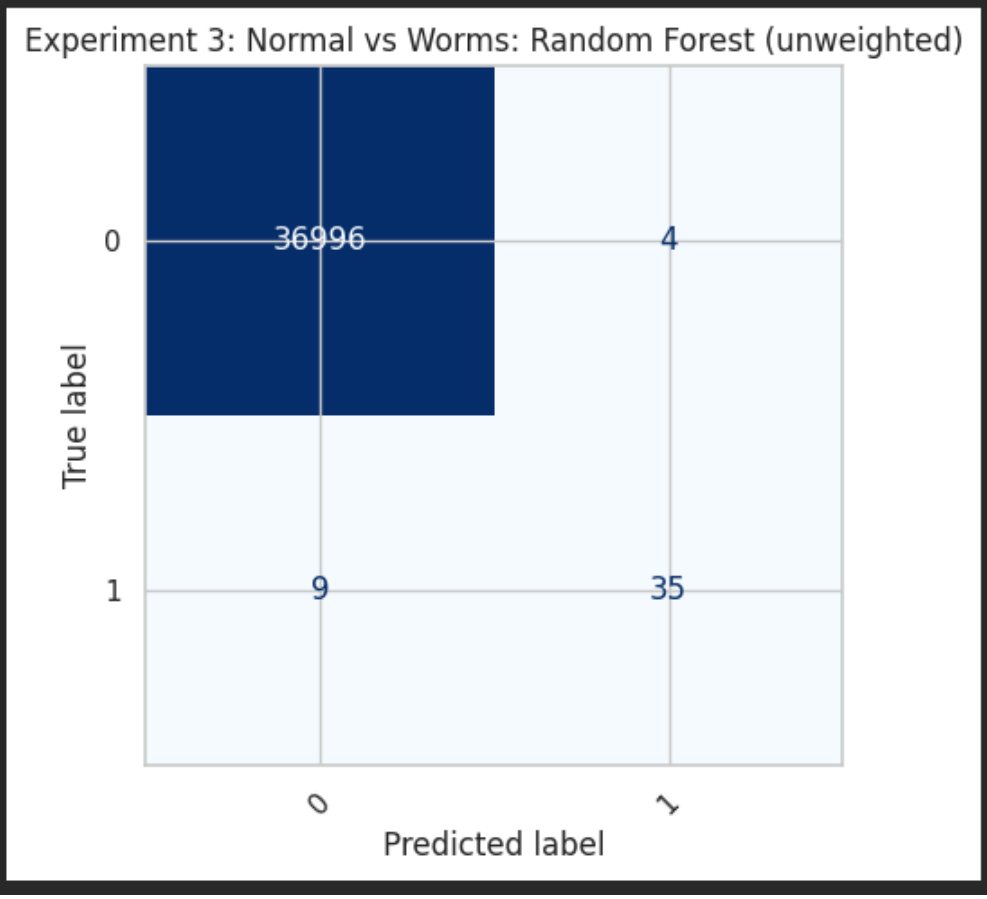


Screenshot 12 — Experiment 3 balanced Logistic Regression.

```
Experiment 3: Normal vs Worms | Random Forest | unweighted
              precision    recall  f1-score   support

           0     0.9998    0.9999    0.9998     37000
           1     0.8974    0.7955    0.8434        44

    accuracy                         0.9996     37044
   macro avg     0.9486    0.8977    0.9216     37044
weighted avg     0.9996    0.9996    0.9996     37044
```

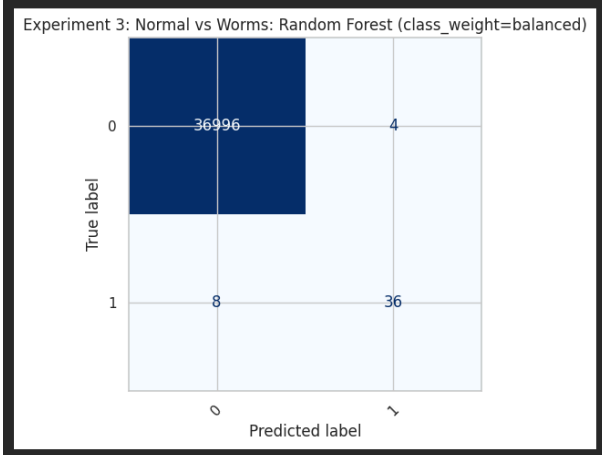


Screenshot 13 — Experiment 3 unweighted Random Forest.

```
Experiment 3: Normal vs Worms | Random Forest | class_weight=balanced
              precision    recall  f1-score   support

           0     0.9998    0.9999    0.9998     37000
           1     0.9000    0.8182    0.8571        44

    accuracy                         0.9997     37044
   macro avg     0.9499    0.9090    0.9285     37044
weighted avg     0.9997    0.9997    0.9997     37044
```



	experiment	model	setting	accuracy	precision_macro	recall_macro	f1_macro	precision_weighted	recall_weighted	f1_weighted	roc_auc
0	Experiment 3: Normal vs Worms	Logistic Regression	unweighted	0.9994	0.8873	0.8522	0.8689	0.9994	0.9994	0.9994	0.9817
1	Experiment 3: Normal vs Worms	Logistic Regression	class_weight=balanced	0.9683	0.5156	0.9160	0.5223	0.9987	0.9683	0.9828	0.9794

Screenshot 14 — Experiment 3 balanced Random Forest and comparison.

Validation search results

	experiment	model	parameters	train_macro_f1	validation_macro_f1	validation_accuracy	validation_roc_auc	overfit_gap_train_minus_validation
1	Experiment 1: multiclass attack_cat	Logistic Regression	{"C": 1.0}	0.4147	0.4056	0.7784	0.9491	0.0091
0	Experiment 1: multiclass attack_cat	Logistic Regression	{"C": 0.3}	0.4093	0.4035	0.7760	0.9484	0.0059
3	Experiment 1: multiclass attack_cat	Random Forest	{"max_depth": null}	0.7979	0.5710	0.8250	0.9037	0.2270
2	Experiment 1: multiclass attack_cat	Random Forest	{"max_depth": 20}	0.6664	0.5588	0.8301	0.9603	0.1076
4	Experiment 2: Normal vs Attack	Logistic Regression	{"C": 0.3}	0.9211	0.9242	0.9369	0.9838	-0.0031
5	Experiment 2: Normal vs Attack	Logistic Regression	{"C": 1.0}	0.9221	0.9241	0.9368	0.9840	-0.0021
7	Experiment 2: Normal vs Attack	Random Forest	{"max_depth": null}	0.9980	0.9542	0.9604	0.9939	0.0438
6	Experiment 2: Normal vs Attack	Random Forest	{"max_depth": 20}	0.9766	0.9505	0.9575	0.9938	0.0261

Screenshot 15 — Validation hyperparameter search and train–validation gaps.

l held-out test results

experiment	model	setting	accuracy	precision_macro	recall_macro	f1_macro	precision_weighted	recall_weighted	f1_weighted	roc_auc_ovr_macro	roc_auc
Experiment 1: multiclass attack_cat	Logistic Regression	selected {"C": 1.0}	0.6981	0.3806	0.3847	0.3365	0.7945	0.6981	0.7060	0.9488	nan
Experiment 1: multiclass attack_cat	Random Forest	selected {"max_depth": null}	0.7543	0.5372	0.4846	0.4660	0.8367	0.7543	0.7756	0.9420	nan
Experiment 2: Normal vs Attack	Logistic Regression	selected {"C": 0.3}	0.8076	0.8492	0.7891	0.7933	0.8393	0.8076	0.7988	nan	0.9550
Experiment 2: Normal vs Attack	Random Forest	selected {"max_depth": null}	0.8711	0.8971	0.8582	0.8649	0.8891	0.8711	0.8678	nan	0.9786
Experiment 3: Normal vs Worms	Logistic Regression	unweighted	0.9994	0.8873	0.8522	0.8689	0.9994	0.9994	0.9994	nan	0.9817
Experiment 3: Normal vs Worms	Logistic Regression	class_weight=balanced	0.9683	0.5156	0.9160	0.5223	0.9987	0.9683	0.9828	nan	0.9794
Experiment 3: Normal vs Worms	Random Forest	unweighted	0.9996	0.9486	0.8977	0.9216	0.9996	0.9996	0.9996	nan	0.9993
Experiment 3: Normal vs Worms	Random Forest	class_weight=balanced	0.9997	0.9499	0.9090	0.9285	0.9997	0.9997	0.9997	nan	0.9986

d: lab01_tuning_results.csv and lab01_final_test_metrics.csv

Screenshot 16 — Final held-out metric table.

```
Saved: lab01_tuning_results.csv and lab01_final_test_metrics.csv
Experiment 1: multiclass attack_cat: highest final macro F1 = 0.4660 (Random Forest, selected {"max_depth": null}).
Experiment 2: Normal vs Attack: highest final macro F1 = 0.8649 (Random Forest, selected {"max_depth": null}).
Experiment 3: Normal vs Worms: highest final macro F1 = 0.9285 (Random Forest, class_weight=balanced).
```

Screenshot 17 — Notebook's automatic final conclusion.